3.1. Регулярные выражения основы

3. Сети, базы данных, безопасность кода

В этом уроке

- Разберемся как искать информацию в тексте
- Изучим базовый синтаксис регулярных выражений
- Научимся использовать модуль re в Python
- Рассмотрим задачи поиска, фильтрации, валидации с помощью regex
- Изучим реальные примеры: поиск IP, email, URL, подозрительных фраз

Регулярные выражения

Регулярные выражения (англ. regular expressions) — формальный язык, используемый в компьютерных программах, работающих с текстом, для поиска и осуществления манипуляций с подстроками в тексте.

Регулярные выражения

Для поиска используется строка-образец (англ. pattern, то есть шаблон (маску), состоящую из символов и метасимволов и задающую правило поиска.

Для манипуляций с текстом дополнительно задаётся строка замены, которая также может содержать в себе специальные символы

Регулярные выражения

Так что такое **регулярные выражения**?

- Специальный язык поиска по шаблону в тексте
- Используется в Python, bash, grep, редакторах кода, СУБД и т. д.
- Позволяет найти, заменить, извлечь фрагменты строк

Регулярные выражения

Типовой запрос

```
import re

pattern = r'\d+'        # шаблон: одна или более цифр
text = 'Уровень: 42%'

match = re.search(pattern, text)
print(match.group())   # → '42'
```

Методы модуля re

```
>>> import re
```

Метод	Суть
re.match(pattern, string, flags=0)¶	Найти строку, начинающуюся с шаблона
re.search(pattern, string, flags=0)	Найти в строке string первую строчку, подходящую под шаблон pattern;
re.fullmatch(pattern, string, flags=0)	Проверить, подходит ли строка string под шаблон pattern;
re.split(pattern, string, maxsplit=0)	Аналог str.split(), только разделение происходит по подстрокам, подходящим под шаблон pattern;
re.findall(pattern, string)	Найти в строке string все непересекающиеся шаблоны pattern;
re.finditer(pattern, string)	Итератор всем непересекающимся шаблонам pattern в строке string (выдаются match-объекты);
re.sub(pattern, repl, string, count=0)	Заменить в строке string все непересекающиеся шаблоны pattern на repl;

https://docs.python.org/3/library/re.html

Регулярные выражения

Поиск всех чисел в тексте:

```
text = "Проверка: 1 ошибка, 2 предупреждения, 0 критичных"
re.findall(r'\d+', text)
# → ['1', '2', '0']
```

Все числа

Создаём шаблоны

Шаблон	Описание	Пример	Применяем к тексту
.	Один любой символ, кроме новой строки \n.	м.л.ко	молоко, малако, Им0л0коИхлеб
\d	Любая цифра	СУ\d\d	СУ35, СУ111, АЛСУ14
\D	Любой символ, кроме цифры	926\D123	926)123, 1926-1234
\s	Любой пробельный символ (пробел, табуляция, конец строки и т.п.)	бор\soда	бор ода, бор, ода, борода
\S	Любой непробельный символ	\S123	X123, я123, !123456, 1 + 123456
\w	Любая буква (то, что может быть частью слова), а также цифры и _	\w\w\w	Год, f_3, qwert
\W	Любая не-буква, не-цифра и не подчёркивание	com\W	com!, com?
[..]	Один из символов в скобках, а также любой символ из диапазона a-b	[0-9][0-9A-Fa-f]	12, 1F, 4B
[^..]	Любой символ, кроме перечисленных	<[^>]>	<1>, <a>, <>>

Создаём шаблоны

Шаблон	Описание	Пример	Применяем к тексту
\d≈[0-9], \D≈[^0-9], \w≈[0-9a-zA-Z a-яA-ЯёЁ], \s≈[\f\n\r\t\v]	Буква “ё” не включается в общий диапазон букв! Вообще говоря, в \d включается всё, что в юникоде помечено как «цифра», а в \w — как буква. Ещё много всего!		
[abc-], [-1]	если нужен минус, его нужно указать последним или первым		
[*[(+\\\]\t]	внутри скобок нужно экранировать только] и \		
\b	Начало или конец слова (слева пусто или не-буква, справа буква и наоборот). В отличие от предыдущих соответствует позиции, а не символу	\bвал	вал, перевал, Перевалка
\B	Не граница слова: либо и слева, и справа буквы, либо и слева, и справа НЕ буквы	\Bвал	перевал, вал, Перевалка

Создаём шаблоны. Квантификаторы

Символ	Значение	Пример	Результат
{n}	Ровно n повторений	\d{4}	1, 12, 123, 1234, 12345
{m,n}	От m до n повторений включительно	\d{2,4}	1, 12, 123, 1234, 12345
{m,}	Не менее m повторений	\d{3,}	1, 12, 123, 1234, 12345
{,n}	Не более n повторений	\d{,2}	1, 12, 123

Регулярные выражения

Поиск URL-адресов

```
text = "Наши сайты: https://site.com, http://test.org, ftp://bad.url"
urls = re.findall(r'https?://[\w\.-]+', text)
# → ['https://site.com', 'http://test.org']
```

URL-адреса

Регулярные выражения

Поиск адресов электронной почты

```
emails = ['ivan@mail.ru', 'test.com', 'user123@gmail.com']
pattern = r'^\w+@\w+\.\w{2,}$'

valid = [e for e in emails if re.match(pattern, e)]
# → ['ivan@mail.ru', 'user123@gmail.com']
```

e-mail

Регулярные выражения

Поиск IP-адресов

```
text = "Попытка входа с IP: 192.168.1.1 и 10.0.0.300"
pattern = r'\b(?:\d{1,3}\.){3}\d{1,3}\b'
ips = re.findall(pattern, text)
# → ['192.168.1.1', '10.0.0.300'] (проверка диапазона — отдельная задача)
```

IP-адреса

Регулярные выражения

Пишем свой антифрод

```
text = "Переведите деньги срочно! Это не шутка!"
pattern = r'\b(переведите|срочно|выигрыш|бонус|не шутка)\b'
matches = re.findall(pattern, text, re.IGNORECASE)
# → ['переведите', 'срочно', 'не шутка']
```

Подозрительные слова

Итоги урока

- Разобрались как искать информацию в тексте
- Изучили базовый синтаксис регулярных выражений
- Научились использовать модуль re
- Рассмотрели примеры использования регулярных выражений